Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments

Christophe Pallier, Julie Bonnaire, and Marie-France Fourcade
Cognitive Neuroimaging Unit
CNRS INSERM CEA

We introduce `goxpyriment`, a new open-source software framework for programming behavioral and cognitive experiments using the Go programming language. The library is designed to address some limitations of existing Python-based experiment tools, particularly the runtime environment complexity that frequently complicates deployment across laboratories. Because Go is a compiled language that can natively embed assets (e.g., graphics, audio files, and stimulus lists), `goxpyriment` compiles entire experiments into single, self-contained executable binaries with zero runtime dependencies. This drastically simplifies distribution to collaborators and testing computers. The programming interface, inspired by Expyriment (Krause & Lindemann, 2014), was designed to be human friendly. The library includes an array of visual stimuli (text, shapes, images, Gabor patches, motion clouds, ...) and audio capabilities (WAV playback and tone generation). While developping `goxpyriment`, we focused on timing reliability. Input events are timestamped by the operating system at hardware-interrupt time, so reaction times are computed by subtracting two OS-level timestamps rather than relying on continuous polling. Go's garbage collector can be disabled, greatly reducing the probability of unpredictable pauses that could corrupt stimulus timing. Finally, a set of over forty psychology experiments implemented in `goxpyriment` are provided that promote not only learning by humans but also improve the ability of modern AI-assisted coding tools to program experiments. The framework is released under the GNU General Public License v3 and is freely available at https://github.com/chrplr/goxpyriment.

Keywords: framework, library, stimulus presentation, timing, behavioral experiments, psychophysics, psychology

Introduction

Experiment-control software occupies a central position in behavioral and cognitive neuroscience: the temporal precision of stimulus delivery and response recording, the ergonomics of experiment programming, and the ease of deploying experiments to laboratory hardware each affect the quality and reproducibility of empirical data. Over the past two decades, Python has become the dominant language for experiment programming, primarily through two libraries: Expyriment (Krause & Lindemann, 2014) and PsychoPy (Peirce et al., 2019). Before Python, MATLAB with the Psychophysics Toolbox (Brainard, 1997; Kleiner et al., 2007) was the de facto standard in many laboratories. More recently, web-based frameworks such as jsPsych (de Leeuw, 2015; de Leeuw et al., 2023) and lab.js (Henninger et al., 2022) have enabled large-scale online data collection.

Each class of tool involves tradeoffs. MATLAB, extended with specialized C/C++ libraries (e.g. the Psychtoolbox), can offer good timing on well-configured hardware. Yet Psychtoolbox scripts written on Windows often do not work out of the box on Linux (and vice-versa). MATLAB can also be quite expensive when one cannot purchase academic licenses (as is the case for non diploma-delivering research institutions). Python tools provide a more open, flexible programming environment with a large scientific ecosystem. Python-based experiments can achieve high-precision timing, for example by relying on the psychtoolbox Python module which provides pieces of the Psychtoolbox-3 (see https://pypi.org/project/psychtoolbox/). However, the use of Python introduces practical difficulties. First, deploying an experiment to a computer that does not already have a compatible Python environment requires package management (conda, pip, venv) and can be a source of breakage across operating-system versions. For example, installing the `psychopy` module under Linux often crashes while attempting to compile the wxPython package when there is no

Christophe Pallier Correspondence should be addressed to C. Pallier, Cognitive Neuroimaging Unit, Neurospin, Gif-sur-Yvette, F-91191, France. E-mail: christophe@pallier.org. I would like to thank Luca Bonatti and Raphael Bordas for alpha testing this software. The authors have no conflicts of interest to disclose.

precompiled wheel. Second, Python's garbage collector can introduce unpredictable pause times that corrupt the timing of stimulus presentation and response recording during the critical trial loop. Web-based tools trade timing precision for accessibility: browser-mediated stimulus presentation is subject to event-loop latency that makes sub-frame timing difficult or impossible (Anwyl-Irvine et al., 2021; Bridges et al., 2020).

The Go programming language (Donovan & Kernighan, 2016) offers a different perspective. Designed from the ground up for simplicity, Go has fewer moving parts than Python; as a consequence, the code is often described as "boring" but highly predictable, as the language structure typically enforces only one way to solve a given problem. In addition, Go is strongly typed and encourages error checking, which translates into less run-time debugging. Another important feature of Go is that it (cross-)compiles to native machine code for each target platform and can produce self-contained binaries that embed the stimuli, experiment lists, and thus have no runtime dependencies. Its garbage collector is concurrent and can be suspended for the duration of a timing-critical section. These properties make it an attractive basis for a behavioral experiment library that prioritizes timing reliability and deployment simplicity.

The present article describes the design and capabilities of `goxpyriment`, discusses its timing architecture, and situates it relative to existing tools. Specifically, we highlight three main contributions of this framework: (1) zero-dependency deployment via single compiled binaries that simplify distribution across laboratory hardware; (2) native high-precision timing achieved directly through the Go and SDL3 APIs without requiring external C/C++ helper engines; and (3) a linear, straightforward API that is well suited for modern AI-assisted coding workflows.

Framework Design

The choice of Go as the implementation language for `goxpyriment` provides several critical advantages over the interpreted languages traditionally used in behavioral research.

Single-Binary Deployment (Zero Dependencies)

Go compiles programs into a single statically linked executable with no external runtime dependencies. Crucially, the `go-sdl3` package embeds the pre-built SDL3 runtime libraries (SDL Developers, 2026) directly into the executable. Stimuli, experimental lists, fonts, and other assets can be embedded too. As a result, distributing an experiment requires copying just one file; there is no need to manage virtual environments, package versions, or interpreter installations across laboratory computers. This substantially reduces the operational overhead of running experiments on multiple machines or sharing materials with collaborators.

Explicit Garbage Collection Control

While the garbage collectors in languages like Python or JavaScript can introduce unpredictable pause times that corrupt timing, Go exposes a runtime API for suspending the garbage collector. `goxpyriment` uses this mechanism in its timing-critical loops, ensuring that stimulus onsets and offsets are not delayed by memory management tasks.

Programming Model

Every experiment centers on an `Experiment` object that manages the window, renderer, and audio subsystems. Execution is wrapped in a `Run` callback that provides a safety layer: if the participant presses the Escape key or closes the window, the library catches the resulting internal signal, safely flushes the data file to disk, and shuts down the SDL subsystems. This ensures that data is preserved even if a session is aborted prematurely.

Experiments follow a hierarchical structure of blocks and trials directly inspired by Expyriment. Each trial is a collection of factors that can be randomized and logged. Data collection is handled by an `exp.Data` object, which writes to a CSV file with extensive metadata headers.

Pre-Experiment Setup Dialog

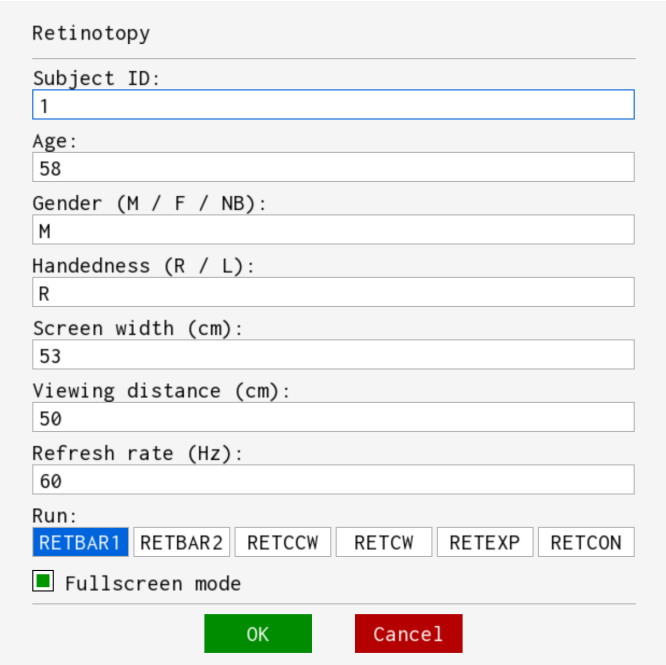
`goxpyriment` provides an optional graphical `GetParticipantInfo` dialog that opens before the main experiment window (see Fig. 1). It allows the experimenter to input subject IDs, select experimental conditions from discrete menus, and toggle settings like fullscreen mode. These values are automatically recorded in the output data file header.

Programs that use `GetParticipantInfo` also accept a `-headless` command-line flag. When present, the dialog is skipped entirely and field defaults (or values previously saved to the cache) are used without opening any window. This is convenient for scripted or automated runs (e.g., continuous integration checks or batch stimulus generation) where interactive input is not possible.

Timing Architecture and Precision

Hardware-Synchronized Presentation

Through go-sdl3 (Jung, 2026), `goxpyriment` relies on the Simple DirectMedia Layer (SDL, version 3), a cross-platform development library designed to provide low-level access to audio, keyboard, mouse, joystick, and graphics hardware (SDL Developers, 2026). For example, `screen.Update()` calls SDL's `SDL_RenderPresent`. When VSYNC is enabled, this function blocks until the display's vertical retrace. When VSYNC is disabled, `SDL_RenderPresent` returns immediately, supporting variable refresh rate (VRR) monitors.



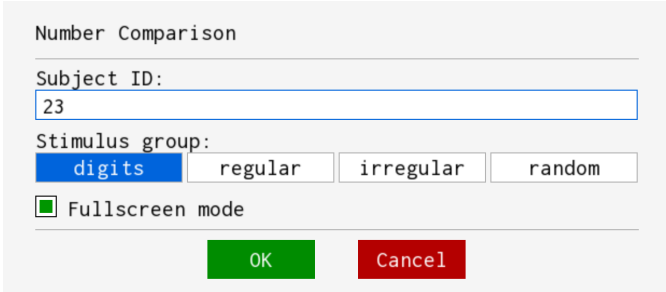


Figure 1

Examples of `GetParticipantInfo` *dialogs for the* Retinotopy *(left) and* Number-Comparison *(right) paradigms provided in the examples accompanying the library. This UI makes it practical for the experimenter to choose among a small fixed set of options without typing, which reduces setup errors when launching an experiment from a desktop shortcut or file manager. Note that which fields appear in the UI is configurable.*

High-Precision Streams

For paradigms with dense, high-speed stimulus sequences (e.g., retinotopic stimulation), `goxpyriment` provides specialized stream functions such as `PresentStreamOfImages`. These functions disable garbage collection, drain the event queue, and busy-wait at the sub-frame level to ensure that each stimulus onset is scheduled accurately. A `TimingLog` records the target and actual onsets for post-hoc verification, as well as the timing of keys pressed during the presentation of the stream.

Hardware-Level Response Timestamps

Response timing in `goxpyriment` leverages SDL3 hardware event timestamps. The `GetKeyEventTS` method returns the `KeyboardEvent.Timestamp` field, which is populated by the OS input subsystem at hardware-interrupt time (nanoseconds since SDL initialization). Because SDL events carry their timestamp in the event structure itself, the event remains in the queue with its original timestamp regardless of how much computation occurs between stimulus onset and response collection. By comparing the response timestamp with the `Screen.FlipTS()` timestamp—which uses the same clock—reaction times can be measured with minimal jitter and no interpreter overhead. This unified timing regime is abstracted through the `ResponseDevice` interface, which explicitly signals whether a timestamp is "precise" (hardware-level) or "non-precise" (polling-based).

The Timing Verification Suite

The repository includes a dedicated timing verification suite (`tests/Timing-Tests/`) that allows researchers to characterize their hardware's performance. The suite includes tests for measuring frame-interval jitter, verifying single-frame flash reliability via photodiode, and calculating audio-visual SOA latencies. Other tests measure hardware-buffer pipeline latency for audio and characterize the USB-serial jitter of trigger devices. A special folder in the repository (`performance-reports`) is intended for users to share the results of tests on their own hardware.

AI-Assisted Coding

An AI-Friendly framework

Go's design emphasizes simplicity and predictability. Because the language has few synonymous constructs, it is particularly well-suited for use with AI-powered coding assistants. The framework's AI-friendliness also comes from other primary technical merits:

Type Safety. Unlike dynamic languages where errors like misspelled method names or invalid argument types (e.g., supplying a string "red" where a RGB color value is expected) only surface at runtime, Go's compiler catches these during the first build attempt. The strict typing reduces erroneous API calls in AI-generated code.

Linear Control Flow. In some other frameworks, the programmer must manage an explicit event loop (e.g., `while True`), manually polling for events and checking for exit signals. AI Coders can miss these steps, leading to non-responsive windows or missed reation times. In `goxpyriment`, the `exp.Run` wrapper handles the event loop and automatic cleanup internally, simplfting the work of both the human and AI coder.

Asset Embedding: A frequent failure point for AI-generated experiments is incorrect file paths for stimuli. Using Go's `//go:embed` directive, an AI can generate code that bundles assets directly into the binary, eliminating path errors caused by mismatched directory structures.

A Cautionary Note

As we just argued, `goxpyriment` is well-suited for AI-assisted coding: its API and the provided examples give strong guidance to AI coding agents prompted to program psychology experiments. Someday experiments from the examples gallery were actually completely entirely generated by Claude or Gemini (see the Author Contributions section). Yet, even if these agents are extremely apt and quick at coding, human supervision remains clearly necessary, as illustrated in Fig. 2.

Comparison with Existing Tools

Table 1 summarizes key properties of `goxpyriment` alongside five widely used experiment frameworks.

`goxpyriment` resembles Expyriment in its API design: a central experiment object, a `present()`-based stimulus API, a structured trial-loop model, and output to `.csv`-format files with metadata. The primary technical differences are the use of Go rather than Python and the use of SDL3 rather than the older Pygame/SDL2 stack. From the user's point of view, `goxpyriment` adds *stream* objects which, as their name indicates, allow displaying sequences of stimuli with tightly controlled timing.

PsychoPy offers a substantially larger stimulus set and a graphical Builder environment; `goxpyriment` has neither of these. However, PsychoPy's greater complexity also makes deployment and version management more challenging. jsPsych enables large-scale online data collection, which `goxpyriment` does not support. The MATLAB Psychophysics Toolbox (PTB) offers the most mature timing validation in the field; `goxpyriment` aims to achieve comparable precision through its native compiled architecture.

Example Experiments

The repository provides over forty examples demonstrating classic paradigms, including the Stroop task, Attentional Blink (RSVP), Masked Priming, Retinotopic mapping for fMRI, and adaptive threshold estimation using the QUEST algorithm. These examples serve as both functional templates and a gallery of the framework's capabilities, ranging from simple parity-judgment tasks to complex moving-display studies.

Timing Results

Limitations and Future Directions

More tests `goxpyriment` The framework has been primarily tested on Linux. Experiments from the Gallery run well on macOS and Windows, as expected given the cross-platform nature of Go and SDL3. But contributors will be neede to run the Timing-Tests suite on these plateforms and add their reports to the performance-reports folder.

Video synchronization. The video playback functions currently do not expose frame-accurate timestamps or hardware trigger outputs. Adding precise onset timestamps and trigger support to video playback is our highest priority for the next release, as these capabilities are required for EEG and fMRI paradigms that include video stimuli.

Eye-tracker support. There is none for the moment. This is clearly a target for future releases.

Online deployment. `goxpyriment` does not currently support web-based deployment. This should be possible in theory, by compiling to web assembly (wasm). We attempted it, but realized that, in practice, the various tools involved are not yet mature enough. This is a real possibility for the future though.

Language barrier. Researchers who are unfamiliar with Go, that is, the vast majority, will need to invest a bit of time in getting acquainted with the language. To this end, many web resources exist, for example, https://go.dev/tour/ and https://gobyexample.com/. As Go is a descendant of C, the syntax will be familiar to people who know a language of the same family, e.g., Java or Javascript. Finally, the examples and the migration guide we provide at https://chrplr.github.io/goxpyriment, are also meant to ease the transition.

Author Contributions

Christophe Pallier: Conceptualization, Software implementation, Documentation and Paper Writing, with assistance from **Claude Sonnet 4.6** and **Gemini 2.5**. Consistent with current editorial norms (Nature Portfolio, 2023; Springer Nature, 2023), Claude and Gemini are listed under Author Contributions rather than as a named author, because

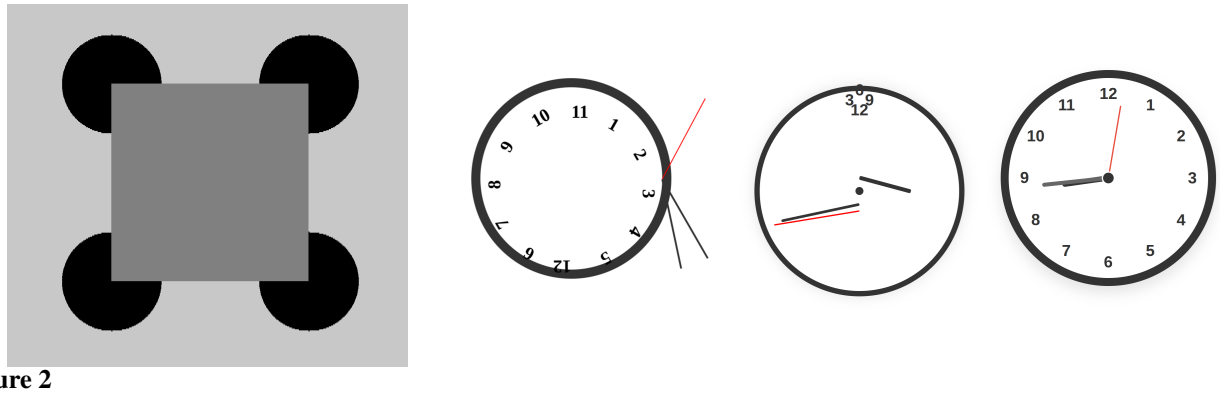


Figure 2

AI errors: *Left: asked to write a program displaying Kanizsa's illusory rectangle, an AI produced the result shown, explaining: "I have changed the Kanizsa example to make things robust and more visible [. . .] A darker gray rectangle is drawn at the center, clearly visible against the light-gray background." Right: output from the AI clock project where AIs are asked to write programs drawing a clock; note the correct response on the right (see https://clocks.brianmoore.com/ for more).*

Table 1

Comparison of selected experiment frameworks.

Property	goxpyriment	Expyriment	PsychoPy	PTB (MATLAB)	jsPsych
Language	Go	Python	Python	MATLAB	JavaScript
Deployment	Single binary	pip/conda	Standalone or pip	MATLAB license	CDN / npm
GC control	Explicit disable	No	No	N/A	No
VSYNC sync	Yes (SDL3)	Yes (SDL2)	Yes (Pyglet/PTB)	Yes	No
VRR support	Yes	No	No	Partial	No
Hardware triggers	Yes	Yes	Yes	Yes	No
Response device API	Unified	Device-specific	Device-specific	Device-specific	N/A
Event timestamp	SDL3 hardware	pygame timer	iohub/PTB clock	`GetSecs` (μs)	Browser clock
Stimulus set	Moderate	Moderate	Extensive	Extensive	Moderate
GUI builder	No	No	Yes (Builder)	No	No
Online deployment	No	No	Partial (PsychoJS)	No	Yes
Reference	This paper	Krause and Lindemann, 2014	Peirce et al., 2019	Brainard, 1997	de Leeuw et al., 2023

authorship requires the capacity to take accountability for the work. Christophe Pallier takes full responsibility for all content. **Julie Bonnaire** performed timing tests and drafted a report available at https://github.com/chrplr/goxpyriment/tree/main/performance-reports. **Marie-France Fourcade** designed and built the photodiode system used to measure visual presentation delays in some of the tests.

Open Practices Statement

The source code of `goxpyriment` is available under the GNU General Public License v3 at https://github.com/chrplr/goxpyriment. All example experiments described in this article are included in the repository. No empirical data are reported in this article.

References

Anwyl-Irvine, A. L., Massonnié, J., Flitton, A., Kirkham, N., & Evershed, J. K. (2021). Gorilla in our midst: An online behavioral experiment builder. *Behavior Research Methods*, *52*(1), 388–407. https://doi.org/10.3758/s13428-019-01237-x

Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, *10*(4), 433–436. https://doi.org/10.1163/156856897X00357

Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, *8*, e9414. https://doi.org/10.7717/peerj.9414

de Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Be-*

havior Research Methods, *47*(1), 1–12. https://doi.org/10.3758/s13428-014-0458-y

de Leeuw, J. R., Gilbert, R. A., & Luchterhandt, B. (2023). jsPsych: Enabling an open-source collaborative ecosystem of behavioral experiments. *Journal of Open Source Software*, *8*(85), 5351. https://doi.org/10.21105/joss.05351

Donovan, A. A. A., & Kernighan, B. W. (2016). *The Go programming language*. Addison-Wesley.

Henninger, F., Shevchenko, Y., Meijer, R. R., Inzlicht, M., & Hilbig, B. E. (2022). Lab.js: A free, open, online study builder. *Behavior Research Methods*, *54*(2), 556–573. https://doi.org/10.3758/s13428-019-01283-5

Jung, B. (2026). SDL3 Bindings for Go [Accessed: 2026-04-01].

Kleiner, M., Brainard, D. H., & Pelli, D. (2007). What's new in Psychtoolbox-3? *Perception*, *36*(ECVP Abstract Supplement), 14.

Krause, F., & Lindemann, O. (2014). Expyriment: A Python library for cognitive and neuroscientific experiments. *Behavior Research Methods*, *46*(2), 416–428. https://doi.org/10.3758/s13428-013-0390-6

Nature Portfolio. (2023). Tools and technologies: Artificial intelligence.

Peirce, J. W., Gray, J. R., Simpson, S., MacAskill, M. R., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, *51*(1), 195–203. https://doi.org/10.3758/s13428-018-01193-y

SDL Developers. (2026). Simple DirectMedia Layer, sdl3 [Accessed: 2026-04-01].

Springer Nature. (2023). Artificial intelligence (AI) policy for authors.